Data Structure with Python (25CS33I) - CIE Practice Test II

RES Polytechnic, Bangalore-91 | CSE Semester III | Model Answers

SECTION I

Q1 a) Dictionary operations (12 marks)

```
def add_update(d, name, marks):
    d[name] = marks                 # add new / update existing

def delete(d, name):
    if name in d:
        del d[name]
    else:
        print("Student not present")

def show(d):
    print("Keys  :", list(d.keys()))
    print("Marks :", list(d.values()))
    print("Items :", list(d.items()))

def iterate(d):
    for n, m in d.items():
        print(n, "->", m)

s = {'Ravi': 72, 'Sita': 85}
add_update(s, 'Arun', 60)           # add
add_update(s, 'Ravi', 80)           # update
print(s)
delete(s, input("Enter name to delete: "))
show(s)
iterate(s)
```

Output:

```
{'Ravi': 80, 'Sita': 85, 'Arun': 60}
Enter name to delete: Sita
Keys  : ['Ravi', 'Arun']
Marks : [80, 60]
Items : [('Ravi', 80), ('Arun', 60)]
Ravi -> 80
Arun -> 60
```

Q1 b) File operations (13 marks)

```
f = open("students.txt", "w")
for n in ["Ravi", "Sita", "Arun", "Meena", "Kiran"]:
    f.write(n + "\n")
f.close()

f = open("students.txt", "r")
print(f.read())
f.close()

f = open("students.txt", "a")
f.write("Anil\nDeepa\n")
f.close()

f = open("students.txt", "r")
print(f.read())
print("Name  :", f.name)
print("Mode  :", f.mode)
print("Closed:", f.closed)
f.close()
print("Closed:", f.closed)
```

Output:

```
Ravi
Sita
Arun
Meena
Kiran

Ravi
```

```
Sita
Arun
Meena
Kiran
Anil
Deepa

Name  : students.txt
Mode  : r
Closed: False
Closed: True
```

Q2 a) Set and Tuple (12 marks)

```
s1 = {10, 20, 30, 40}
s2 = {30, 40, 50}
print("Union:", s1 | s2)
print("Intersection:", s1 & s2)
print("Difference:", s1 - s2)
print("Symmetric Diff:", s1 ^ s2)

t = (1, 2, 3, 4, 5)
print(t[0], t[-1])                  # indexing
print(t[1:4])                       # slicing

p = ("Vinay", 22, "Bangalore")      # packing
name, age, city = p                 # unpacking
print(name, age, city)
```

Output:

```
Union: {10, 20, 30, 40, 50}
Intersection: {30, 40}
Difference: {10, 20}
Symmetric Diff: {10, 20, 50}
1 5
(2, 3, 4)
Vinay 22 Bangalore
```

Q2 b) List and String (13 marks)

```
l = [5, 12, 8, 3, 20, 7]
print(l, l[0], l[-1], l[1:4])

l.append(15)
l.remove(int(input("Enter number to remove: ")))
l.sort()
print(l)

sq = [x * x for x in l if x % 2 == 0]
print("Squares of even:", sq)

s = "Python"
print(s[0], s[-1], s[1:4])
print(s + " Lang")                  # concatenation
print(s * 2)                        # repetition
print(s.upper(), s.lower(), s.replace("P", "J"))
```

Output:

```
[5, 12, 8, 3, 20, 7] 5 7 [12, 8, 3]
Enter number to remove: 3
[5, 7, 8, 12, 15, 20]
Squares of even: [64, 144, 400]
P n yth
Python Lang
PythonPython
PYTHON python Jython
```

SECTION II

Q3 a) Exception handling (12 marks)

```
try:
    a = int(input("Enter first number: "))
    b = int(input("Enter second number: "))
    r = a / b
except ZeroDivisionError:
    print("Cannot divide by zero")
except ValueError:
    print("Invalid input, enter numbers only")
else:
    print("Result =", r)
finally:
    print("Execution completed")
```

Output:

```
Enter first number: 10
Enter second number: 0
Cannot divide by zero
Execution completed
```

Error type	Example	Reason
Syntax error	`print("Hello"`	Closing bracket missing
Runtime error	`print(10/0)`	ZeroDivisionError occurs while running
Logical error	`avg = a + b / 2`	Should be (a+b)/2; wrong answer, no error

Q3 b) Factorial: recursion vs iteration (13 marks)

```
def fact_rec(n):
    if n == 0 or n == 1:            # base case
        return 1
    return n * fact_rec(n - 1)      # recursive case

def fact_itr(n):
    f = 1
    for i in range(1, n + 1):
        f *= i
    return f

n = int(input("Enter number: "))
print("Recursion factorial:", fact_rec(n))
print("Iteration factorial:", fact_itr(n))
```

Output:

```
Enter number: 5
Recursion factorial: 120
Iteration factorial: 120
```

Difference: Recursion - function calls itself (uses call stack, more memory). Iteration - uses a loop (faster, less memory). Both give the same result (120).

Q4 a) Student class - User Defined Data Type (12 marks)

```
class Student:
    def __init__(self, name, roll, marks):
        self.name = name
        self.roll = roll
        self.marks = marks

    def display(self):
        print("Name:", self.name, "| Roll:", self.roll, "| Marks:", self.marks)

    def update_marks(self, m):
        self.marks = m

s1 = Student("Ravi", 1, 72)
s2 = Student("Sita", 2, 85)
s1.display()
s2.display()
s1.update_marks(90)
s1.display()
```

Output:

```
Name: Ravi | Roll: 1 | Marks: 72
Name: Sita | Roll: 2 | Marks: 85
Name: Ravi | Roll: 1 | Marks: 90
```

Q4 b) List ADT using class (13 marks)

```
class ListADT:
    def __init__(self):
        self.items = []

    def insert(self, item):
        self.items.append(item)

    def delete(self, item):
        if item in self.items:
            self.items.remove(item)
        else:
            print(item, "not present")

    def search(self, item):
        if item in self.items:
            print(item, "is present")
        else:
            print(item, "is not present")

    def display(self):
        print(self.items)

L = ListADT()
L.insert(10)
L.insert(20)
L.insert(30)
L.display()
L.search(20)
L.delete(20)
L.delete(99)
L.display()
```

Output:

```
[10, 20, 30]
20 is present
99 not present
[10, 30]
```

Note: In the exam, attempt one full question from each section (Q1 or Q2, Q3 or Q4), run the code and show the output.